

JavaScript Runtime, Event Loop, and Execution Order

1. What is the JavaScript call stack and how does it process synchronous code?

The Call Stack is a fundamental mechanism that the JavaScript engine uses to track the execution of the program. It operates on a last in first out basis.

Processing of synchronous code:

When a javascript file begins executing,

It creates a global execution context and pushes it inside the empty stack

Then starts pushing other contents of the code line by line.

When a function is called, a new **function execution context** is created and pushed to the top of the stack.

The javascript engine always focuses on the context at the very top of the stack.

When a function finishes execution, it is popped off the stack, and control returns to the execution context immediately below it.

It processes everything sequentially in this order and finally popping the global execution context, emptying the stack and indicating the completion

2. What are microtasks and macrotasks, and how are they scheduled?

Macrotasks (The Task Queue):

Macro tasks represent the standard asynchronous operations which are initiated by the environment apis.

These operations run inside the main thread one by one but only after both the callback queue and the microtask queue are empty.

Common Sources: setTimeout(), setInterval(),

Scheduling: When a macro timer expires or an event occurs, its callback is appended to the Macrotask Queue.

Microtasks(The MicroTask Queue):

These are high priority tasks executed immediately after the callback queue is empty.

Common Sources: Promise callbacks (.then(), .catch(), .finally()) and queueMicrotask()

Scheduling: When a promise settles, its reaction callback is appended to the Microtask Queue.

3. Why Promises, queueMicrotask(), and setTimeout() behave differently in execution order?

They all behave differently because the event loop classifies each of these asynchronous operations into separate categories such as macro and micro tasks.

Promises, queueMicrotask() - micro tasks, these are prioritized first by the event loop and executed before the macro tasks such as setTimeout().

setTimeout() - macro tasks, these are less priority tasks and the event loop executes these tasks only after both the call stack and the microtask queue becomes empty

4. How does the event loop decide what runs when?

The event loop is a continuous process which checks both the call stack and the task queues to decide which operation should run next.

Its operational algorithm can be described as below.

It first looks at the call stack and waits for it to become empty. Then it takes the jobs from the micro task queue and executes them one by one till they become empty. Then it goes to the macro task queue and takes only one oldest job, puts it on the call stack and executes it. Then it checks the micro task queue again to see if any high priority tasks are available and waiting. If no such processes are available, then it continues with the macro tasks. It repeats this process until all of the stack and the queues become empty.

Exercise 1 – Microtasks vs Macrotasks

Code:

```
console.log('1');

setTimeout(() => {
  console.log('2');
}, 0);
```

```
Promise.resolve().then(() => {  
  console.log('3');  
  queueMicrotask(() => {  
    console.log('4');  
  });  
  console.log('5');  
});
```

Predicted output order:

1
3
5
4
2

The 1st line is pushed onto the call stack first, executed, the output is printed and it is popped off the call stack.

Then the `setTimeout()` is pushed and executed inside the call stack, and as it is a macro task, the task is deferred to the browser's web api and we move to the other task of our code. When this deferred task is completed, we add their callback into our macro task queue. for now this 2 is skipped and will be added to the macro task queue when the time comes.

Now we move to the promises section and this is pushed and executed inside the call stack and as the promise is resolved, the promise reaction callback is added to the micro task queue, but not printed yet.

Now the callback queue is empty.

The event loop looks at the micro task queue next. It sees it is not empty and executes the callback. The 3 is printed and the statement is popped. The `queueMicrotask()` adds the 4 inside the microtask queue, but not prints yet. The 5 is getting printed now. Still the microtask queue is not empty, and the 4 gets printed now.

Finally the event loop goes to the macro task queue and executes the 2 inside it and prints it.

Exercise 2 – Nested and Sequential Tasks

Code:

```
console.log('start');  
setTimeout(() => {  
  console.log('setTimeout 1');
```

```
    Promise.resolve().then(() => console.log('Promise 1'));  
  }, 0);
```

```
  setTimeout(() => {  
    console.log('setTimeout 2');  
  }, 0);
```

```
  Promise.resolve().then(() => console.log('Promise 2'));  
  console.log('end');
```

Predicted Output:

```
start  
end  
Promise 2  
setTimeout 1  
Promise 1  
setTimeout 2
```

The 1st console.log line is pushed onto the call stack first, executed, the output is printed and it is popped off the call stack.

Then the first setTimeout() is pushed and executed inside the call stack, and as it is a macro task, the task is deferred to the browser's web api and we move to the other task of our code. When this deferred task is completed, its callback (containing setTimeout 1 and the nested promise) is added into our macro task queue. For now, this is skipped and stays in the macrotask queue.

Then the second setTimeout() is pushed and executed inside the call stack, and as it is also a macro task, this task is also deferred to the browser's web api. When completed, its callback (containing setTimeout 2) is added into our macro task queue, sitting right behind the first one. This is also skipped for now.

Now we move to the promises section Promise.resolve().then(...) and this is pushed and executed inside the call stack. As the promise is resolved, the promise reaction callback (containing Promise 2) is added to the micro task queue, but not printed yet.

Now the last line console.log('end') is pushed onto the call stack, executed, the output is printed and it is popped off.

Now the initial main script execution is complete, and the call stack is empty.

The event loop looks at the micro task queue next. It sees it is not empty and executes the callback. Promise 2 is printed and the statement is popped.

Now that the microtask queue is completely empty, the event loop goes to the macro task queue. It takes the first available macrotask, the callback for the first timeout—and pushes it into the call stack.

Inside this callback, `console.log('setTimeout 1')` is executed and printed. Right after, a new promise section is encountered and executed inside the call stack. As this promise is resolved, its reaction callback (containing Promise 1) is added to the micro task queue. This first macrotask is now complete and popped off the call stack.

Before looking at the next macrotask, the event loop must check the micro task queue again. It sees it is not empty because of the callback we just added. It executes it immediately, and Promise 1 is printed.

Now that the microtask queue is empty again, the event loop returns to the macro task queue. It picks up the next remaining macrotask, the callback for the second timeout and pushes it into the call stack.

Inside this callback, `console.log('setTimeout 2')` is executed, printed, and popped off. Both queues and the call stack are now completely empty, and the execution finishes.

Key Takeaways

Execution order:

Synchronous code is executed first inside the call stack one by one. After them, asynchronous code inside the micro task queue will be executed one by one. The event loop then takes one oldest macro task queue, and looks at the micro task queue to check for any high priority tasks. If no such tasks exist, then it will continue to execute macro tasks. These steps will be followed till all the task queues and the call stack become empty.

Async behavior:

JavaScript is single threaded but still when it encounters an asynchronous task like `setTimeout()`, it performs it in a non blocking way by offloading these tasks to the web api. The web api runs these tasks in the background and when completed, adds the callback to the queues.

How it affects real projects:

If all those asynchronous tasks are performed on the main thread of the javascript, then our application will block the user during that time period. The javascript runtime enables the asynchronous behaviour giving a smooth experience to our users.

